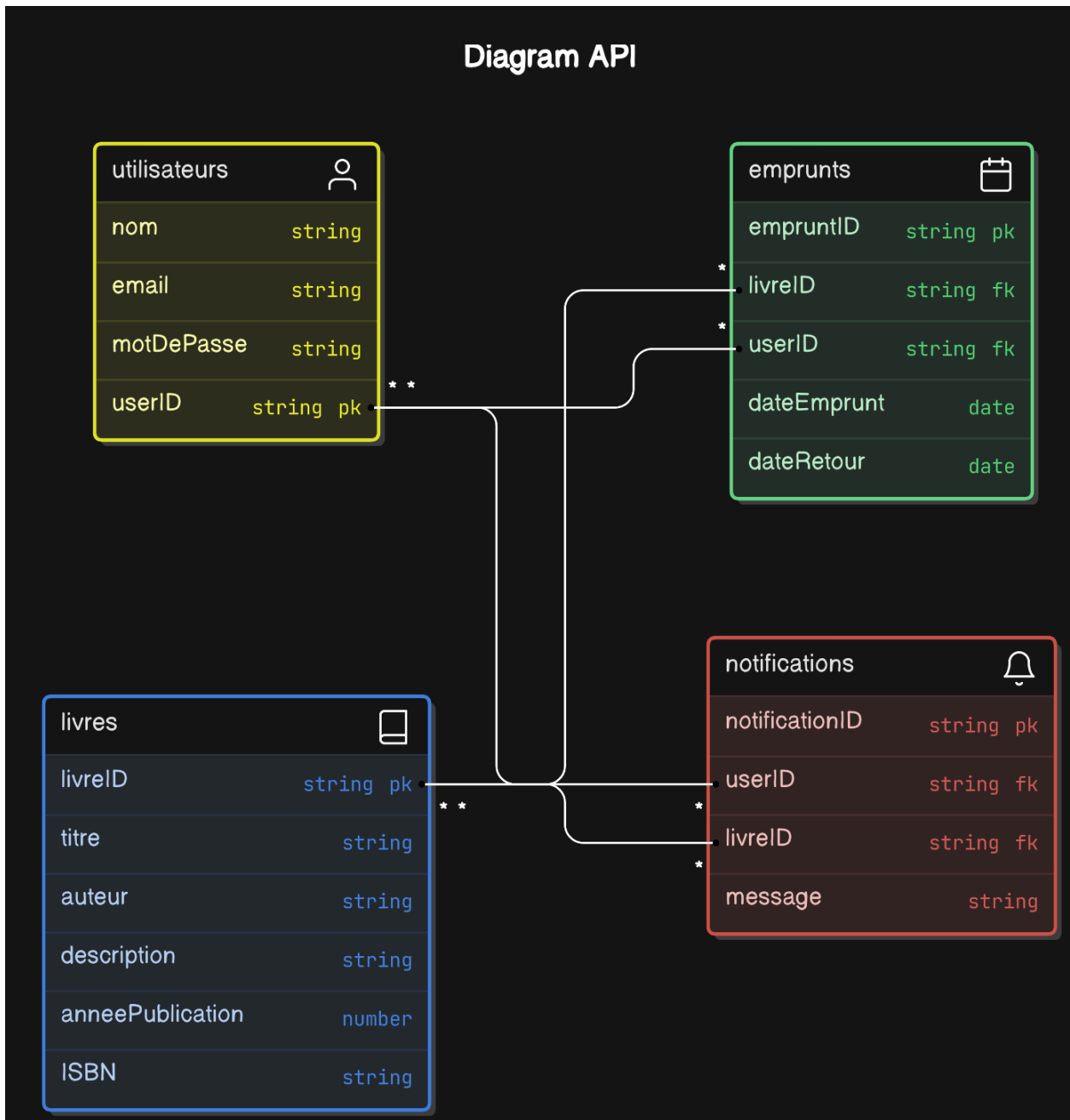

Fiche de Travaux : Node Js - typescript

Création d'une API REST pour une Bibliothèque en Ligne
avec Gestion des Utilisateurs et Notifications

Scénario

En tant que développeur travaillant pour une bibliothèque en ligne qui souhaite permettre aux utilisateurs de consulter des livres, de les emprunter, de les retourner et de recevoir des notifications. Vous devez créer une API REST qui gère ces fonctionnalités avec une gestion avancée des utilisateurs et des notifications. L'API doit permettre les opérations suivantes :

Diagramme :



Fonctions à Implémenter

I. Gestion des Utilisateurs :

A. Inscription d'un nouvel utilisateur :

1. Endpoint : **POST /users/signup**
2. Données d'entrée : nom, email, mot de passe
3. Action : Crée un nouvel utilisateur dans la base de données, chiffre le mot de passe.

B. Connexion des utilisateurs :

1. Endpoint : **POST /users/login**
2. Données d'entrée : email, mot de passe
3. Action : Authentifie l'utilisateur et génère un token JWT.

C. Déconnexion des utilisateurs :

1. Endpoint : **POST /users/logout**
2. Action : Invalide le token de l'utilisateur.

D. Consulter le profil utilisateur :

1. Endpoint : **GET /users/profile**
2. Action : Retourne les informations du profil de l'utilisateur connecté.

E. Mettre à jour le profil utilisateur :

1. Endpoint : **PUT /users/profile**
2. Données d'entrée : nom, email, mot de passe
3. Action : Met à jour les informations du profil de l'utilisateur connecté.

F. Supprimer un compte utilisateur :

1. Endpoint : **DELETE /users/profile**
2. Action : Supprime le compte de l'utilisateur connecté.

II. Gestion des Livres :

A. Consulter la liste des livres disponibles :

1. Endpoint : **GET /books**
2. Action : Retourne la liste de tous les livres disponibles dans la bibliothèque.

B. Ajouter un nouveau livre :

1. Endpoint : **POST /books**

2. Données d'entrée : titre, auteur, description, année de publication, ISBN

3. Action : Ajoute un nouveau livre à la bibliothèque.

C. Mettre à jour les informations d'un livre :

1. Endpoint : **PUT /books/:id**

2. Données d'entrée : titre, auteur, description, année de publication, ISBN

3. Action : Met à jour les informations d'un livre existant.

D. Supprimer un livre :

1. Endpoint : **DELETE /books/:id**

2. Action : Supprime un livre de la bibliothèque.

III. Gestion des Emprunts :

A. Emprunter un livre :

1. Endpoint : **POST /loans**

2. Données d'entrée : livreID, utilisateurID

3. Action : Crée un emprunt pour un livre par un utilisateur, met à jour l'état du livre à "emprunté".

B. Retourner un livre emprunté :

1. Endpoint : **PUT /loans/:id/return**

2. Action : Met à jour l'état du livre à "disponible", marque l'emprunt comme terminé.

C. Consulter l'historique des emprunts d'un utilisateur :

1. Endpoint : **GET /loans/user/:userID**

2. Action : Retourne l'historique des emprunts d'un utilisateur spécifique.

IV. Notifications :

A. Envoyer une notification lorsqu'un livre réservé devient disponible :

1. Action : Lorsqu'un livre réservé par un utilisateur devient disponible, envoie une notification par email à cet utilisateur.

B. Envoyer des rappels pour les dates de retour des livres :

-
1. Action : Envoie des emails de rappel aux utilisateurs pour les informer des dates de retour des livres empruntés.

V. **Stack :**

1. **Node JS**
2. **Express**
3. **Typescript**
4. **Git et Github**
5. **MongoDB**
6. **Prisma ORM**

